Task 3 - NumPy Data Processing

Task 3.1 - Outlier Detection and Correction

I denne oppgaven bruker vi NumPy til å finne priser som avviker betydelig fra gjennomsnittet.

Vi definerer en outliner som en pris som ligger mer enn tre standaravvik fra gjennomsnittet. Disse prisene skal erstattes med medianen av de gyldige prisene.

In [4]:
```python
import numpy as np

def correct_outliers(prices: np.ndarray) -> np.ndarray:

    """
    Finner priser som avviker mer enn tre standardavvik fra gjennomsnittet
    og erstatter dem med madianen av de gyldige prisene.

    Args: (prices: np.ndarray): En array med priser

    Returns: np.ndarray: En array med korrigerte priser.
    """
    prices = np.array(prices, dtype=float, copy=True)

     # Returnerer en tom array dersom ingen priser er oppgitt
    if prices.size == 0:
        return prices

    # Bergener gjennomsnittet og standardavvik
    mean = np.mean(prices)
    std = np.std(prices)

    # Finner priser som ligger mer enn tre standardavvik fra gjennomsnittet
    outliers = np.abs(prices - mean) > (3 * std)

    # Finner prisene som ikke er outliers
    valid_prices = prices[~outliers]

    # Bergener medianen av de gyldige prisene
    median = np.median(valid_prices)

    # Erstatter outliers med medianen
    prices[outliers] = median

    return prices
```

In [5]:
```
# Lager flere vanlige priser og en unormalt høy pris
test_prices = np.array([30] * 20 + [500], dtype=float)

# Korrigerer prisene
resultat = correct_outliers(test_prices)

print("Opprinnelig siste pris:", test_prices[-1])
print("Korrigert siste pris:", resultat[-1])
```

```
Opprinnelig siste pris: 500.0
Korrigert siste pris: 30.0
```

Task 3.2 - Refund ratio

In [3]:
```
def compute_refund_ratio(sales_count: np.ndarray, refund_count: np.ndarray) -> n
    """
    Calculate the element-wise refund ratio using NumPy arrays.
    Adds a small epsilon to prevent division by zero.
    Returns an array containing the calculated refund ratios.
    """

    # Small value to prevent division by zero
    epsilon = 1e-6

    # Calculate refund ratio for each element
    ratio = refund_count / (sales_count + epsilon)

    return ratio


# Test with zero sales
sales = np.array([100, 0, 20])
refunds = np.array([5, 0, 1])

result = compute_refund_ratio(sales, refunds)

print(result)
```

```
[0.05 0.   0.05]
```

Task 3.2 - Refund ratio

The function `compute_refund_ratio()` calculates the refund ratio for each element using NumPy array operations. A small epsilon value (`1e-6`) is added to the sales count to prevent division by zero. The function returns a NumPy array containing the calculated refund ratios.